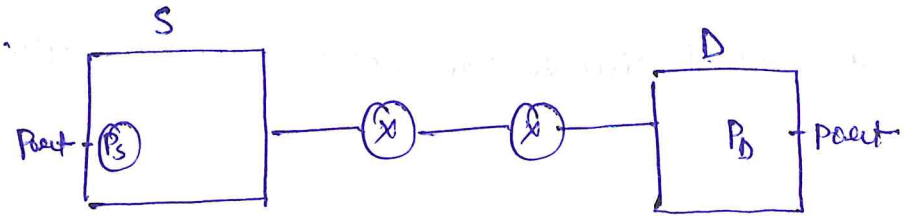
Transport - Layer. (TCP, UDP)

→ Process to process communication.

→ Reliable end-end channel.



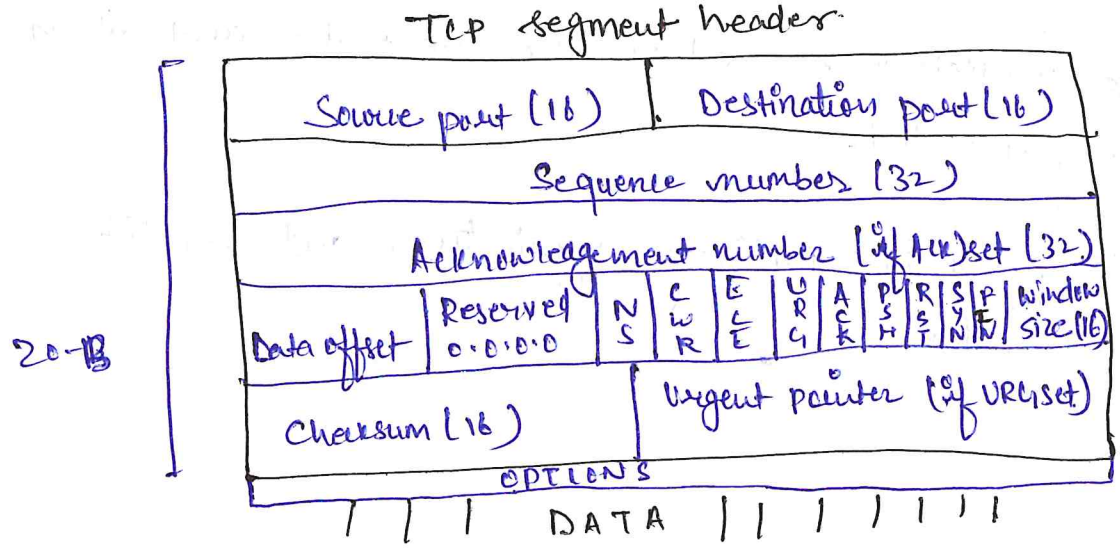
– TCP and UDP

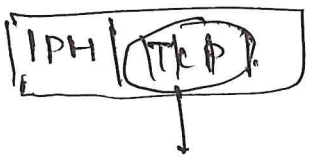
TCP → Connection-oriented | reliable

UDP → Connectionless

Transport layer gives you process to process delivery.

TCP - segment format





TCP = header + data.

* Segment is the term used for transport layer. for packet.

Source and destination port (16 bit)

→ 0 to $2^{16}-1$ = 65535 [Total ports possible]

→ Pre-assigned ports

↓

[0 - 1023) → Predefined ports, Universal ports, fixed ports

IANA

[HTTP : 80
FTP : 21
SMTP : 20
DNS : 25
SSH : 22]

→ 1024 - 49151 → reserved for future

→ 49152 - 65535 → Private-ports, Dynamic ports.

↳ These ports used by Network programmers

Network Socket → Java / python [video-chat]

* This is [IP: port] → Socket is the combinations of IP and PORT.

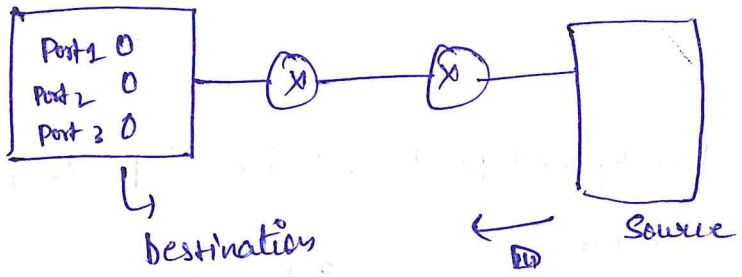
↓

socket

* IP address uniquely identify the computer, port address uniquely identify the process.

Socket → 12.13.14.16 : 4956 2

IP address — Port number



We need a port to reach the exact process because IP address will lead you to the destination after that we need a port to reach the exact process.

Sequence No. and Ack No.

IP: Packet stream [Identification number]

* IP cares about the sequence of packets.

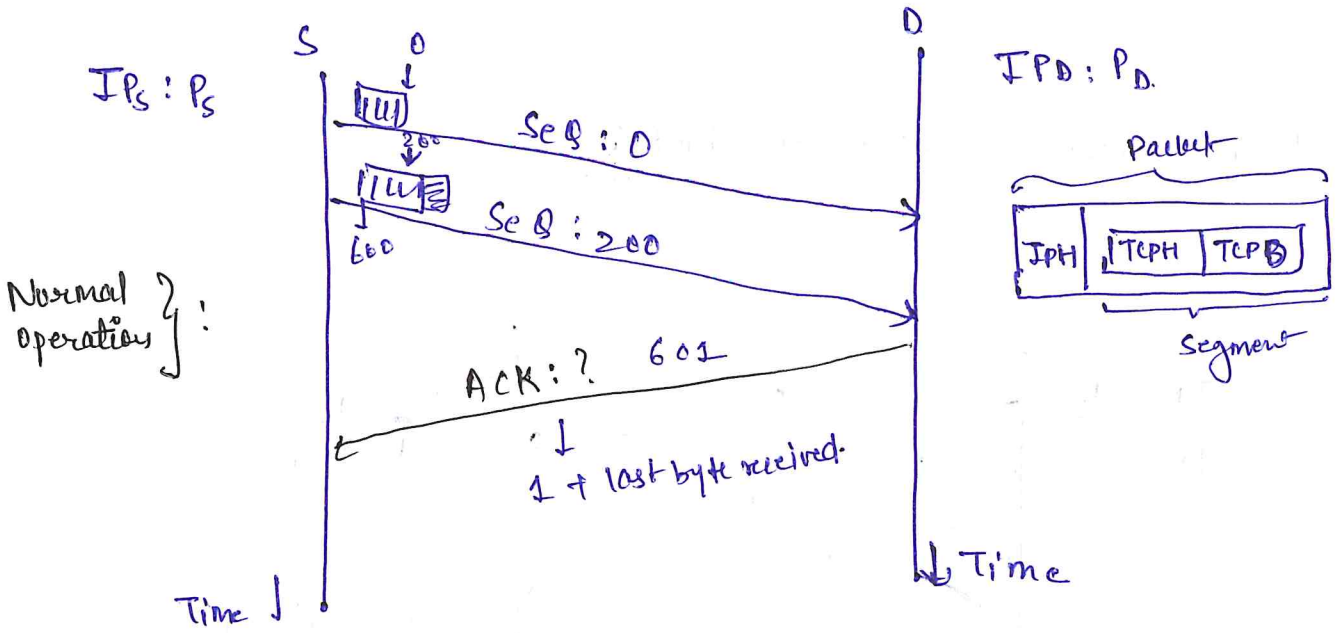
TCP: Byte stream

→ TCP cares about sequence of bytes



Sequence no: = First byte in a segment

* In the transport layer the general term we are using is a segment.



SEQ 0 : first byte of header SEQ 200 : first byte of data.

* TCP is the reliable service and we also have acknowledgement in TCP.

* If last byte of data is 600 then ACK will be 601.

* Computing ACK: TCP packet enclosed under IP packet.

TCP data length = IP packet length − IP header length − TCP header length

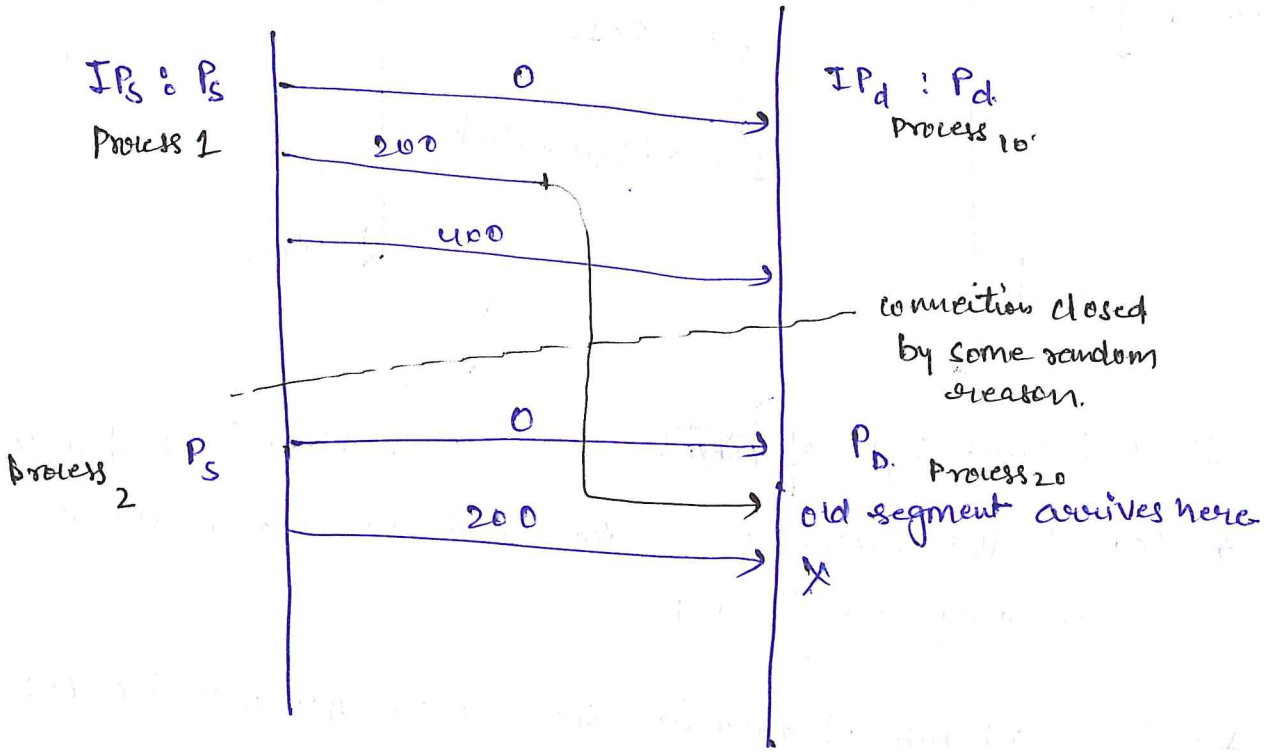
IP packet length: We will get by Total length bits in IP header

IP header length: Using data offset field in TCP header

ACK = TCP data length + 1.

* The way we computed the tcp data length by use of IP header.

Random sequence number as start:

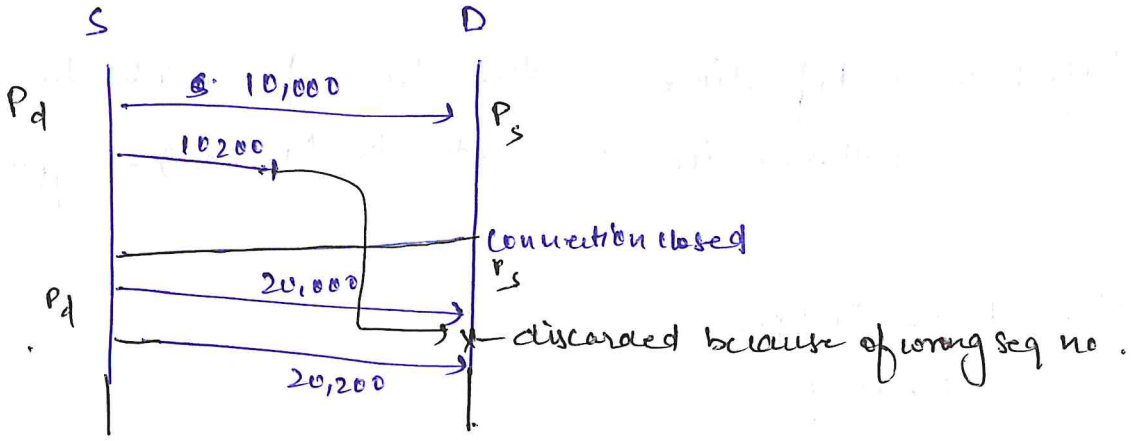


Here our packet with sequence number 200 get delayed and afterwards connection closed because of some reason.

The new connection is established by same port number Coincidentally on both the sides.

The delayed packet in the older connection came and accepted as correct packet because the port number of previous connection and new connection are same.

To avoid this problem we use the concept of random number.



* for starting sequence number instead of giving the number zero. we can give the sequence no. 10,000 randomly.

* The probability of choosing the same sequence number by both the connection is very low.

* $0 - 2^{32} - 1$ (Seq number)

WRAP AROUND SEQ NUMBER:

If we are starting without sequence no. 10,000

Seq no = 2^{32} = 4GB data we can send.

So, if our data is more than 4GB. How will we send because we don't have much seq number?

Solution: WAT = wrap around time.

10,000, 10001, 10002 ---- $2^{32}-1$, 0, 1, ---- 10,000

WAT

Again we start with zero.

* WAT is starting from that sequence number again come to that sequence number.

Life time → Life time is the time for which the packet will remain in network (120sec) → Typically.

Que: Bw = 1MBps

WAT = ?

Ans:

1 MB → 1 sec

2^{20} B → 1 sec

1 B → $1/2^{20}$ sec

Total seq no. ← 2^{32} B → $\frac{2^{32}}{2^{20}}$ = 2^{12} sec = 4K sec = WAT

from starting to come again

WAT > LT

4000 > 120 ✓

* If WAT > LT then there would be no confusion of sequence number.

Eg: Bw = 1GBps

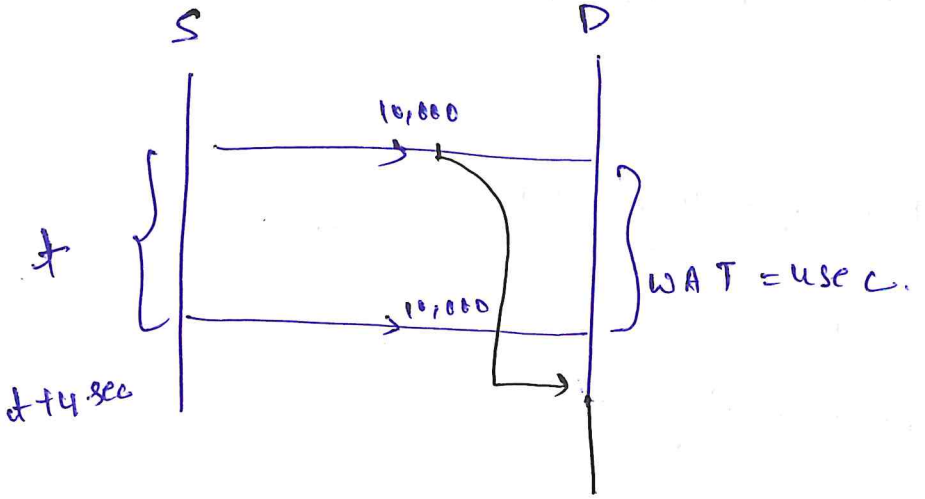
WAT = ?

1GB = 1sec

$1B = \frac{1}{2^{30}}$ sec

$2^{32} B = \frac{2^{32}}{2^{30}} = 4$ sec = WAT

WAT < LT
4 < 180



Problem:

If WAT < LT then after 4 sec that packet again come in the network with same sequence number.

Solution: Increase the number of bits in sequence number.

LT = 180sec

then WAT > 180 sec

1sec = 1GB

180GB = 180GB

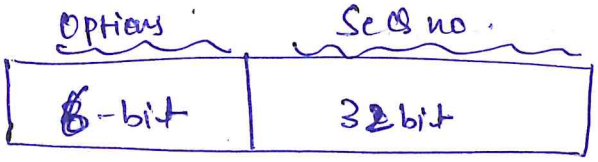
Hence 180 x G sequence number

$= 180 \times 2^{30}$

↓ 8-bit to represent this

↳ 30-bit for this

* Total we need 38 bit



* So, least significant 32-bit will store in sequence number. and 6-bit will represent in options.

→ for WAT < LT we can use this idea.

Header length / Data-offset (4b) : TCP header

→ Just like IPV4

- HL x 4 = header bytes

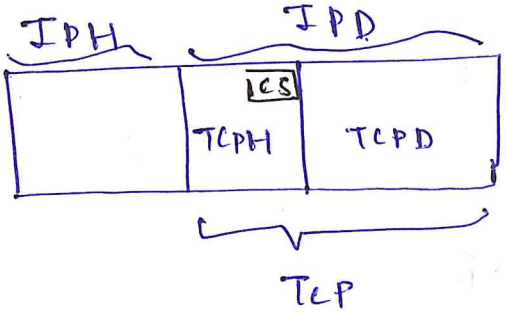
→ min header = 20B

max header = 60B (max HLf = 1111)

The minimum value of data offset = 5 because min size of header = 5 x 4 = 20B

So, if data offset > 5 then we have options

Checksum (16 bit) :-

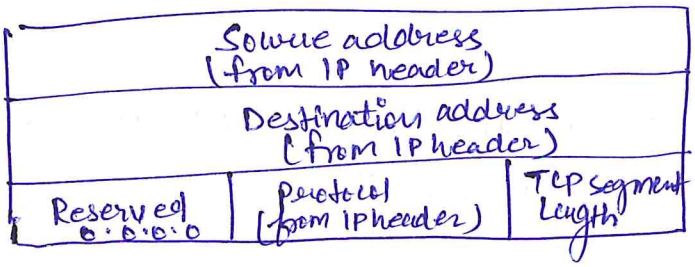


TCP checksum = 16-bit checksum computed using CRC

TCP checksum = It is computed on TCPH + TCPD + pseudo header*

↳ Three parts.

Pseudo header →



"TCP pseudo header" for checksum calculation

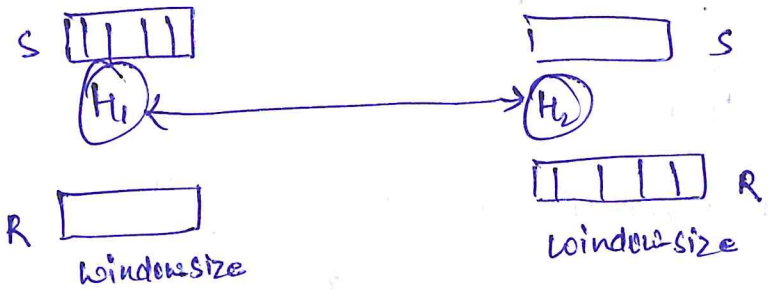
* Here take some important fields in the pseudo header from the Ip header.

→ In IPv4 the checksum is calculated for header not for whole packet.

→ TCP checksum also uses pseudo header for double checking for misdelievery of any packet should not happen.

TCP connection :-

→ connection oriented protocol



TCP enables the full duplex and buffers.

Buffers → some preallocated memory

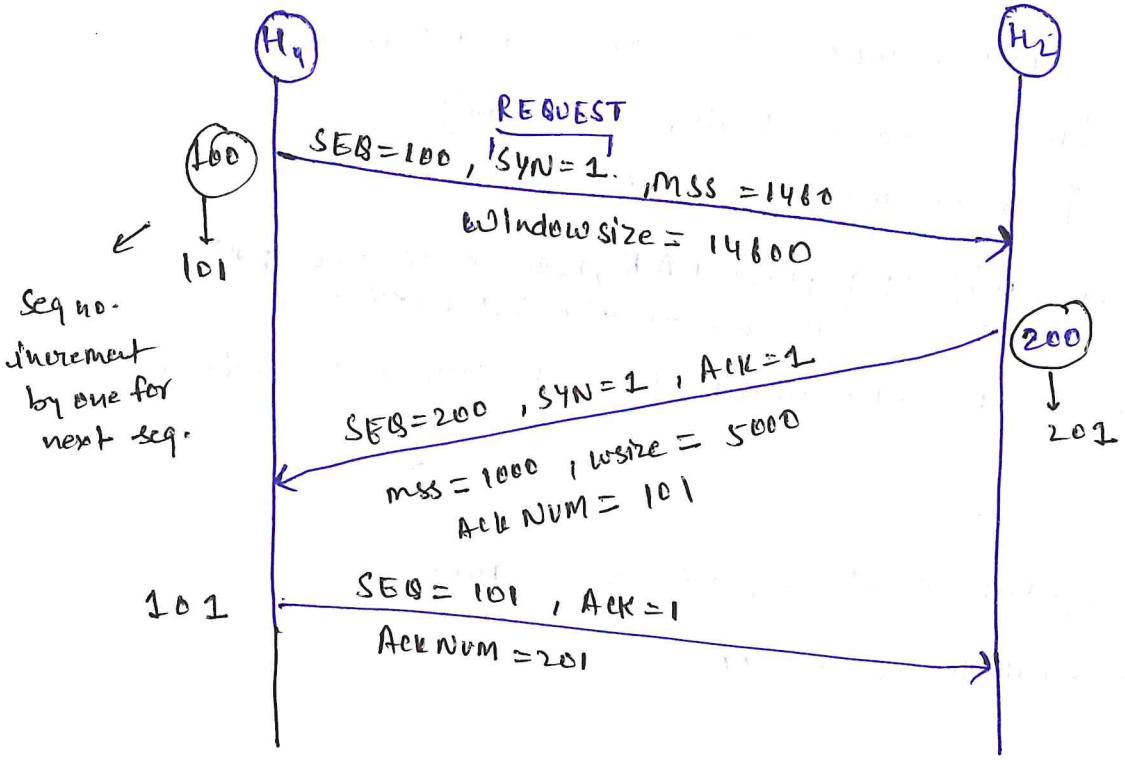
Sender buffer → Stores the bytes which we are transmitting on channel.
or
Receiver buffer

Three steges of TCP :-

- → establishment
- → Transfer
- → Termination

Establishment: 3-way Handshake.

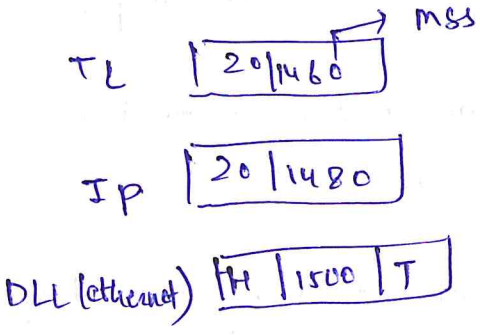
Three way handshake are used for establishing a connection.



* Host 1 sends an packet to H_2 with random seq num = 100 and SYN = 1

MSS (Maximum segment size) → we place mss in options part

→ we are sending from H_1 to H_2, H_1 can be on Ethernet and H_2 can be on another protocol.



* H_1 is informing H_2 that MSS I can handle is 1460 Bytes only.

* By sending window size from H_1 to H_2. H_1 is saying that the maxm window size = 14600 buffers I have to handle.

→ H_2 replies with random SEQ = 200 and SYN = 1

ACK = 1 [means do read the Ack-num]

ACK = 0 [means do not read Ack-num]

Ack NUM = (101) → piggybacked Ack, because we are putting the Ack in the SYN packet.

SYN = 1 means H_2 also want to establish a connection with

→ H_2 also computes maxm Segment size = 1000

and window size = 5000

So, Smaller of [Sw, Rw) will be the size of window established on both sides.

SYN = 1 → we increment the seq no by 1 for the next seq.

ACK = 1 → we increment the seq no. by zero for the next seq.

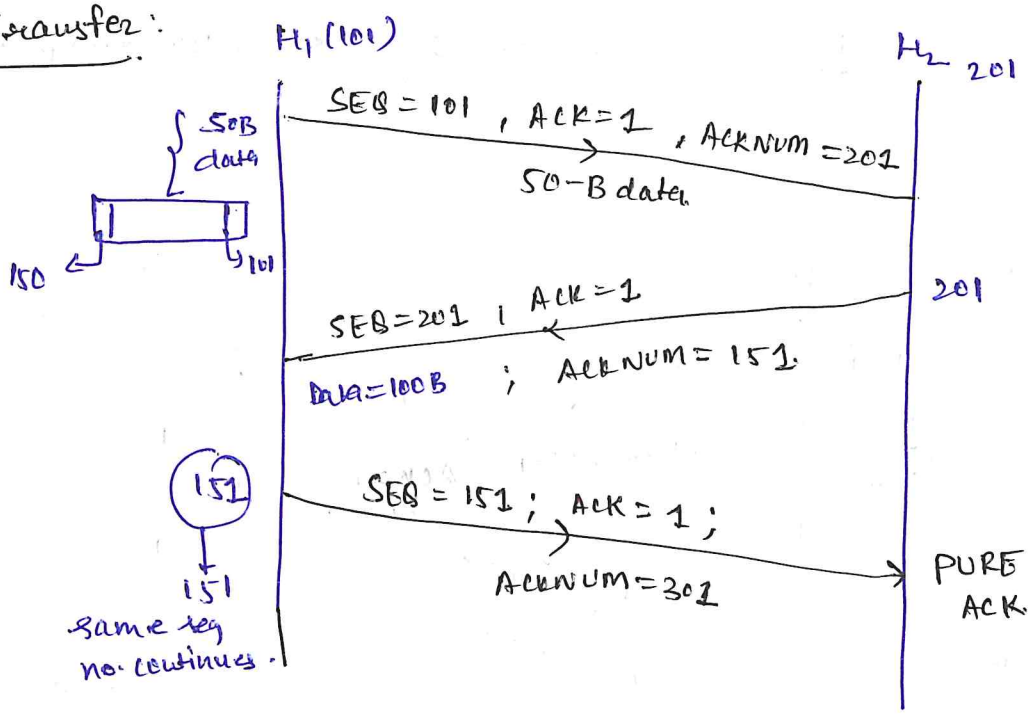
FIN = 1 → we increment the seq no. by one for the next seq.

One Data byte → we increment the seq no by one for the next seq.

→ Host 1 acknowledges the Ack from H_2 and replies with SEQ = 101, ACK = 1, Ack-num = 201

The whole process is called the three way handshake.

Data Transfer:



* In data transfer after connection establishment we are using SEQ = 101 further [because last packet sent in the connection establishment is Ack and we do not increment seq no. for Ack]

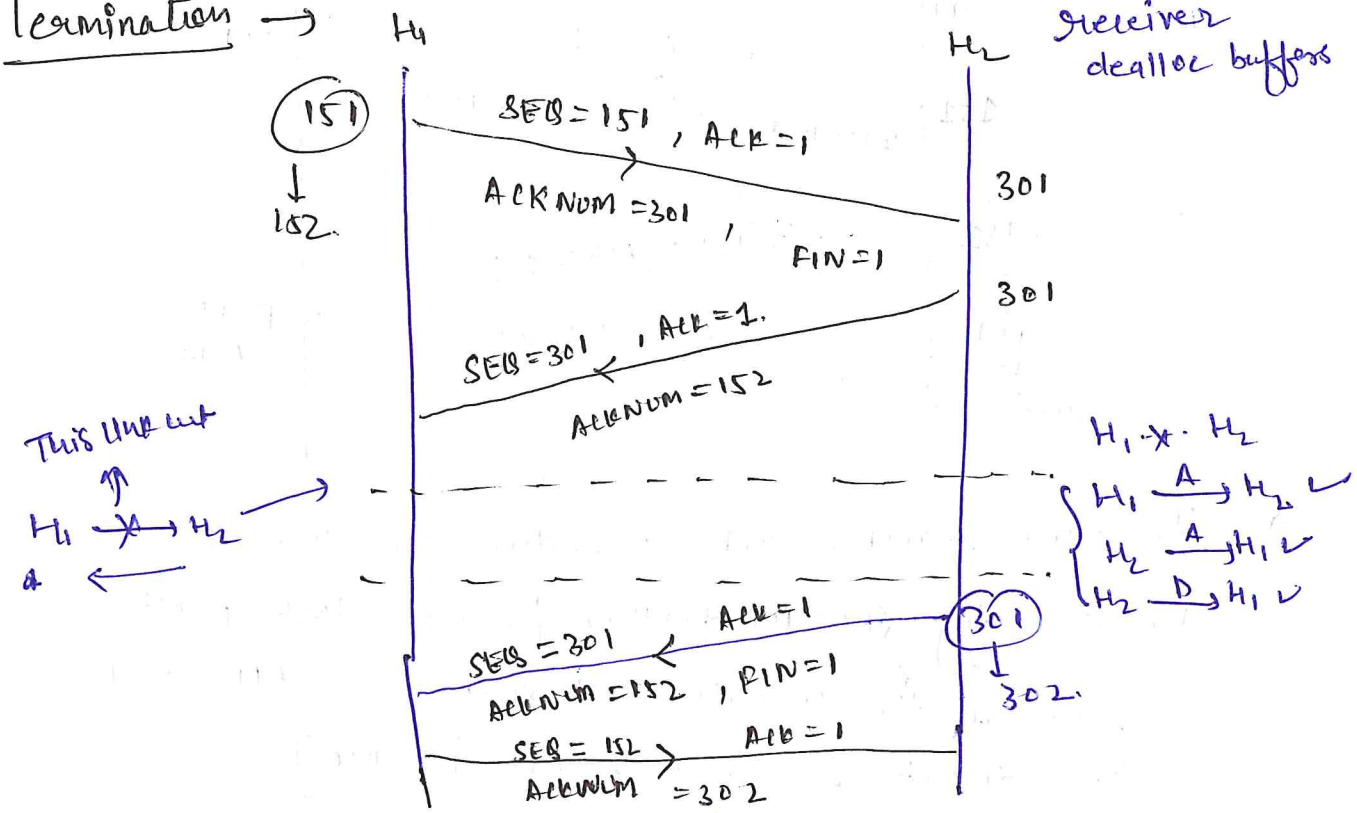
→ we are again sending Ack = 201 for fault tolerance if previous Ack might have lost.

→ The H_2 sends SEQ = 201, ACK = 151 because we have ~~sent~~ received 50 B data

→ Now if we don't have data from H_1 to H_2 we will send Pure Ack with seq no = 151, ACK = 1 Ack Num = 301.

SYN	ACK	
1	0	→ REQUEST (1st)
1	1	→ REPLY (2nd) segment.
0	1	→ Other segments [like data packets]

Termination →



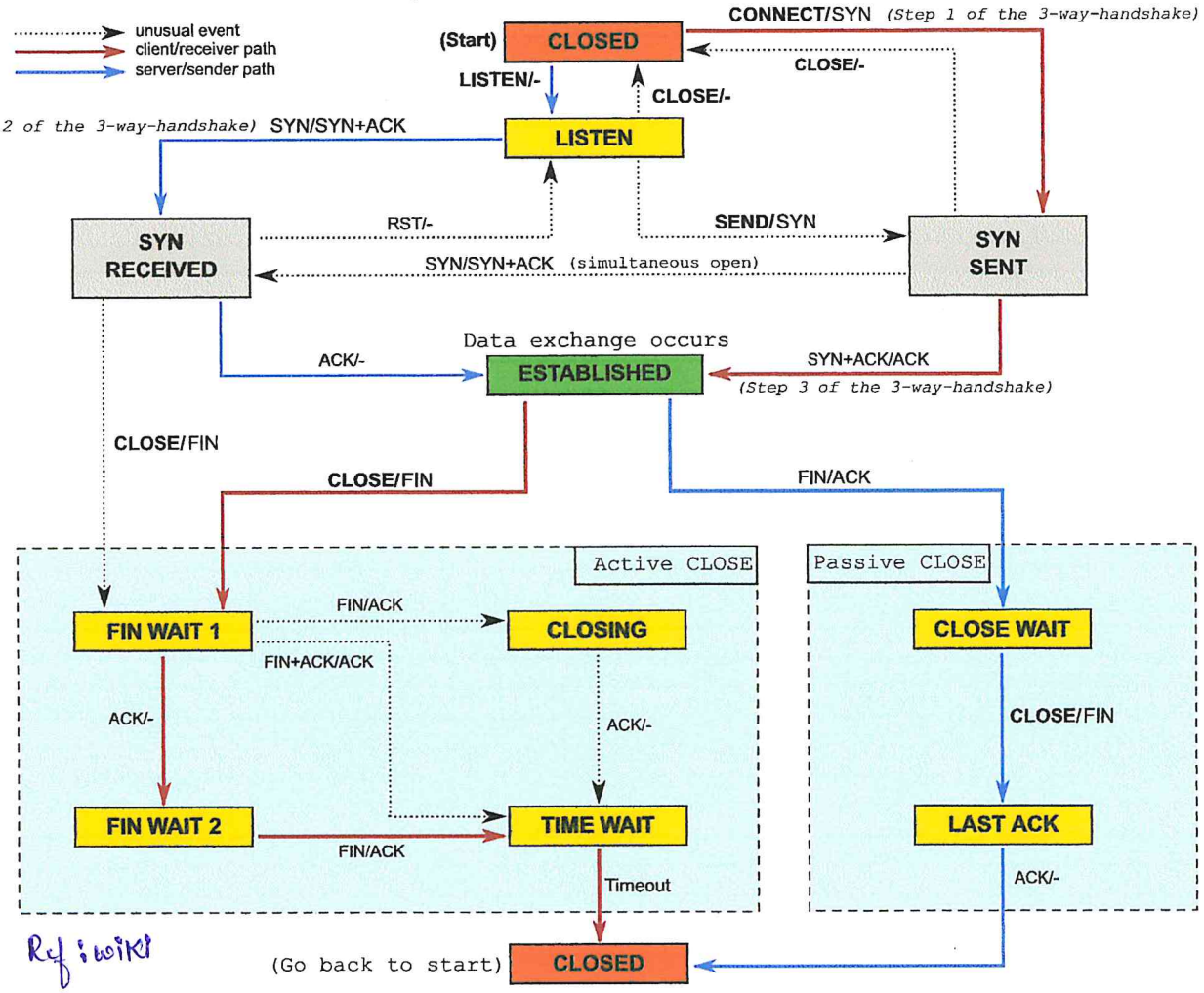
→ By sending FIN = 1 H_1 informing H_2 that it want to terminate the connection.

→ After receiving the Ack from H_2. The link between H_2 to H_1 cut. Now H_1 can not send data.
but H_2 can send data to H_1

→ H_2 also send FIN = 1 for closing the connection

→ Worst case : 4 packets

State Transition diagram :-



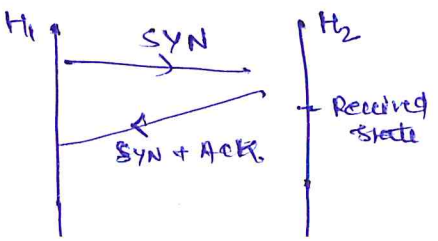
Ref: wiki

* First everything in closed state.
* The brown lines represent the Action corresponding to client, the blue lines correspond the action. Corresponds to server, dotted lines are unusual events.

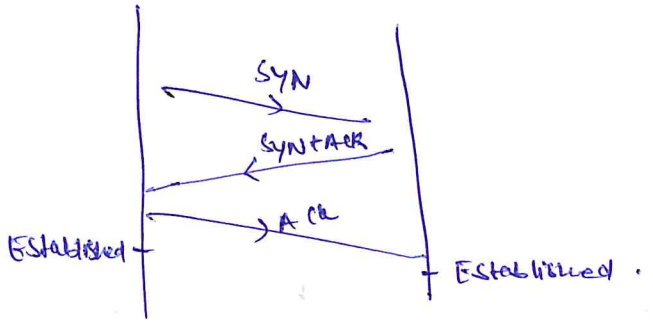
→ Client in closed state and then uses the connect system call and sends a packet SYN=1 request packet.

→ The server is in Listen state and client is in SYN sent state.

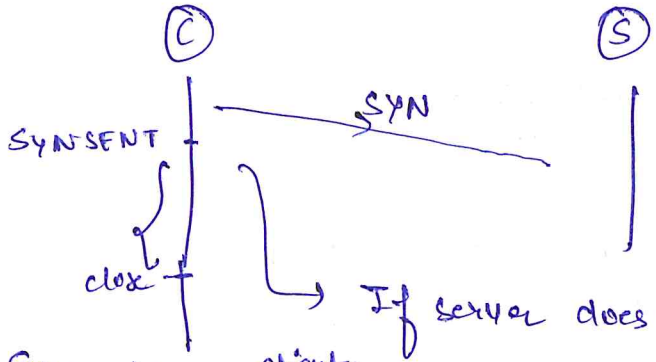
→ The server sent the SYN+ACK and goes to received state.



→ Now sender sent the ACK and both of them went to connection establishment state.



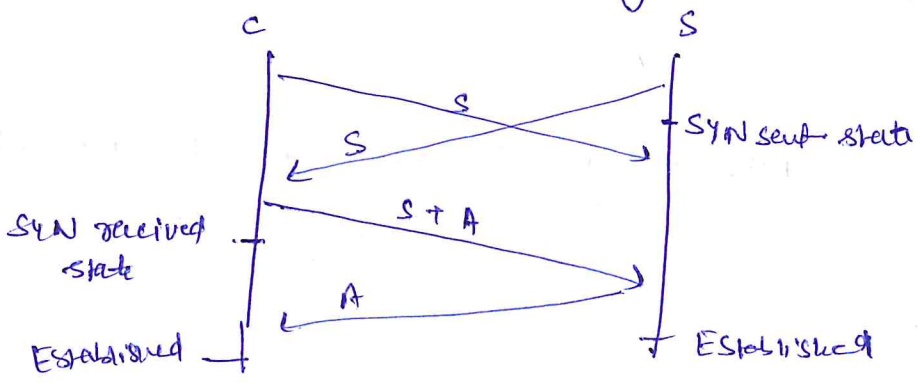
Now understand unusual events: —



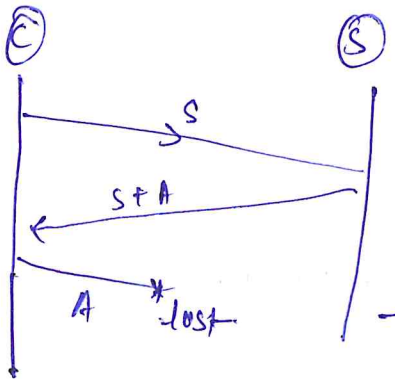
If server does not respond to the SYN, then client closes the connection and client re-initiates the connection.

→ If server in the listen state and does not get SYN from long time then it also closes the connection.

Another event: — If both server and sender sent the SYN packet together [Simultaneous open]



Another unusual events :-



→ Server wait for some time and Reset the whole thing and go to Listen state.

Connection closed mechanism :-

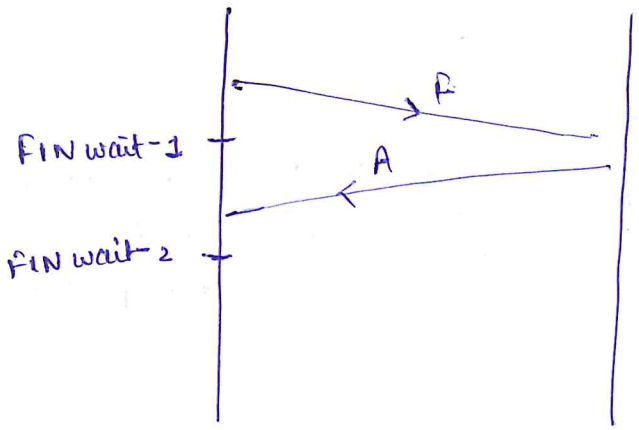
→ Client sends the close signal with FIN packet and goes to FIN wait-1 state.

→ Server sends the Ack in response and goes to close wait state.

* There is two types of close : (a) Active close
(b) Passive close.

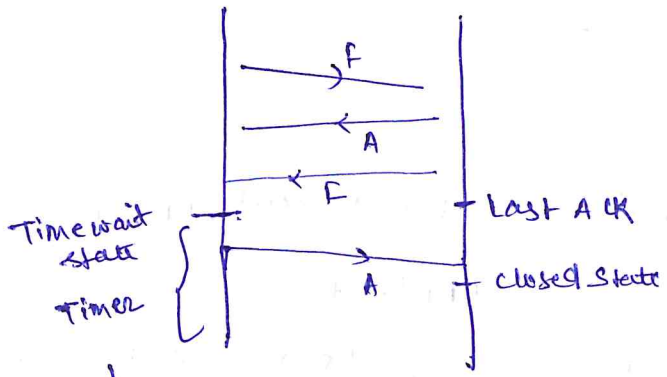
→ Who ever initiates the close Request will go to active close either client or server and other will go to passive close.

→ Client receives an Ack. and goes to client goes to FIN wait 2 stat



Now connection from client to server is closed c ⇒ s and client is waiting for FIN state from server.

→ Server sends the FIN packet, client recives the FIN segment and goes to time wait state.

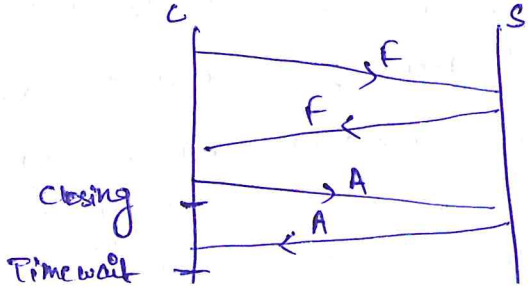


↳ client waits for a few seconds if any packet is coming from server. After timeout it goes to closed state.

I Unusual events:

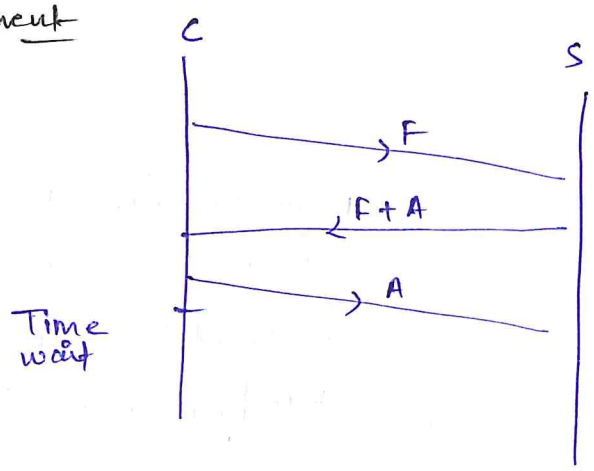
→ Let suppose server is in SYN-received state and wants to close the connection because of some reasons.

II Unusual event → Both client and server sends a FIN



→ Simultaneous closing

14. Unusual event

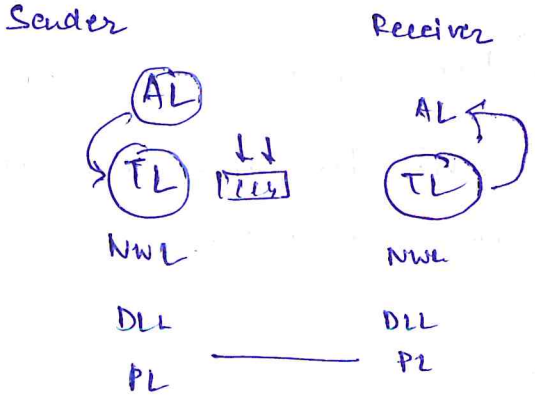


→ If client receives F+A in the same packet

* He can close in less packets also by piggybacking.
* Worst case we need 4 packets to close.

Other fields in TCP header:-

PSH flag: → Max segment size. [don't wait upto MSS]
→ Real time [video/audio/RDP/chat]
→ Push data to Network layer (S) → Sender Size
→ Push data to AL → Receiver Size

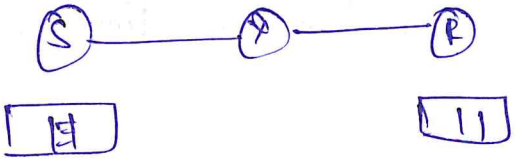


→ The maximum segment size sender and receiver let suppose negotiated with 1000B. If PSH=1 you don't wait 1000B to come from AL to TL to package a segment. whatever data you have [<1000B] also you push down and send to Network Layer.

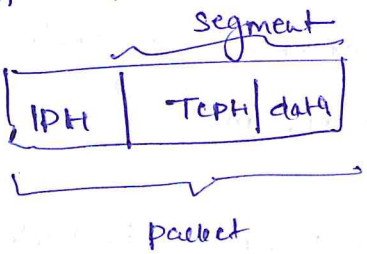
URG and Urgent pointer → (Out of order packet)

→ Send data urgently to the destination.

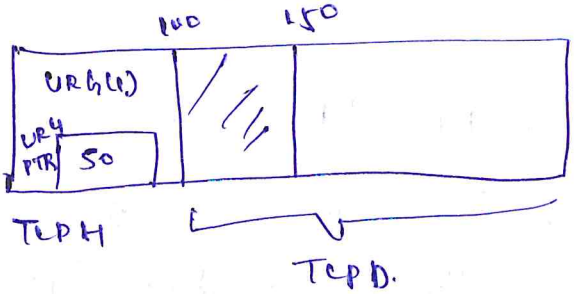
* PSH sends the data in-order while URG can send the data out of order.



→ As routers is the network layer device which can not read the TCP header. So it won't be able to read the URG in TCP header.

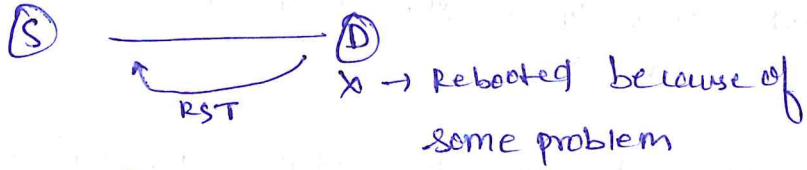


Solution for this problem is In Ipheader we have the priority field where we set priority = 7 (max) for higher priority.



* If URG=1 and URG PTR=50. It means that we have 50 bytes in the whole packet needed to be transfer urgently from SEQ=100 to SEQ=150

RST FLAG :- Close the connection due to unexpected errors.



→ In such condition the destination looses the track and close the connection.

→ If Destination gets the unexpected sequence number way higher then expected also RST the connection.

Options :-

(1) WATCLT ⇒ option field has extra bits (Timestamp).
↳ we have studied earlier.

(2) Max segment size : Parameter negotiation do to fix the mss.

(3) Padding : {header length must be multiple of 4}.
if less fill padding.

(4) window size :- (Parameter negotiation.)

16b ⇒ 2^{16} = 64 KB (too small)

∵ We have 16 bits in window size. If we want bigger window size then we can take some bits from options

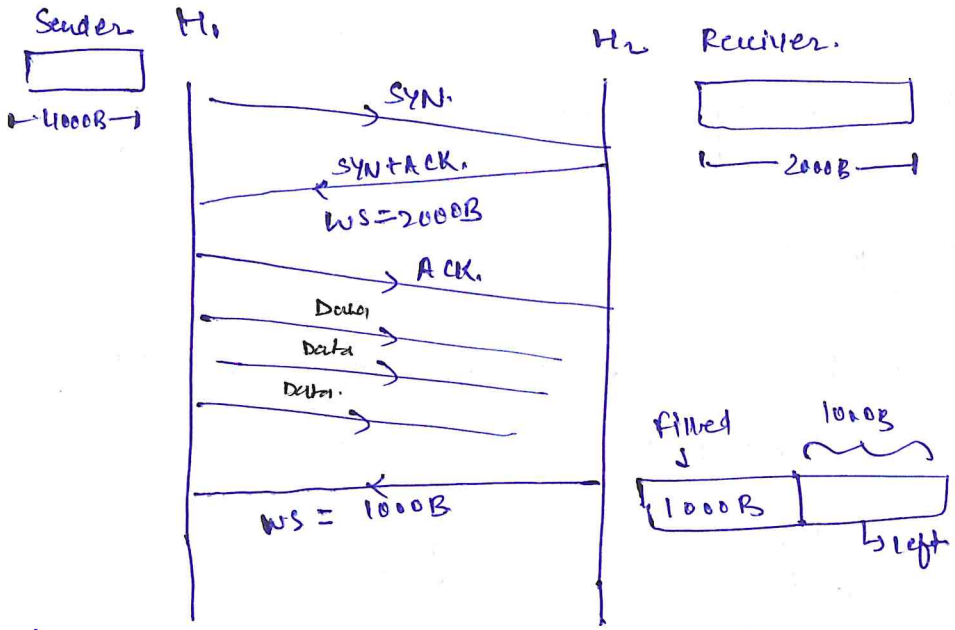
30 b ⇒ 1 GB [Good window size]
16 b | 14 b
window size | (Options : Extended window size)

Flow control :—

The control the rate and flow of data.

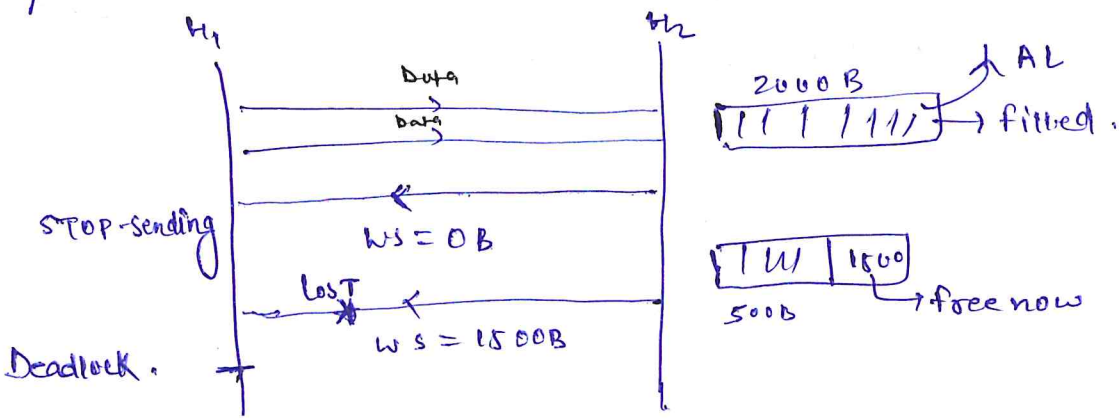
→ Window scaling

→ advertised window size. → This term is similar to window size we read earlier.



* In three way handshake. we are sending the window size. which is also called the advertised window size. because sender and receiver advertising its window size which each other.

* The Receiver again advertised with the window size = 1000B because buffer of receiver is filled and now it is left with only 1000B.



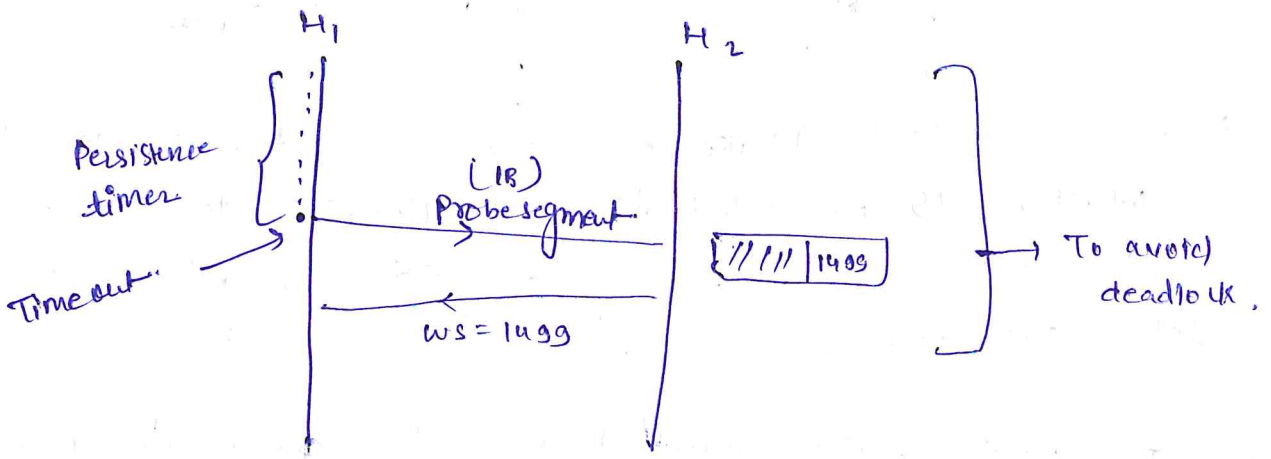
→ When receiver window size is filled. It relpy with WS = 0B to the sender.

→ The sender is waiting for onew window size. After some time when receiver buffered get empty.

→ The Receives again advertised with WS = 1500B and if this packet is lost. This situation is deadlock. because the sender is waiting for window size which receiver has already sent and receiver is waiting for data.

To solve this problem:—

As soon as the sender receives the WS = 0B. The sender will start the persistence timer.



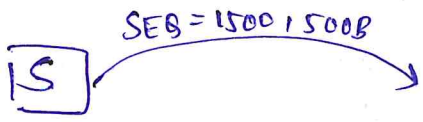
The probe segment of 1B is send by sender and receiver than send the WS = 1499 B because 1B used by buffer.

Sliding window and Re-transmission :-

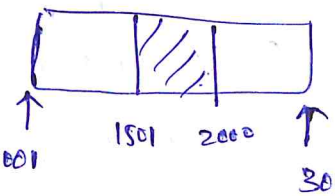
TCP (Hybrid)
- Selective Repeat → Sender ws = Receiver ws, accepts out of order.
- Go-Back-N → Cummulative Ack.

→ TCP uses the hybrid of both.

In TCP: * SW = RW
* It also accepts out of order
* It also uses accumulative Ack.



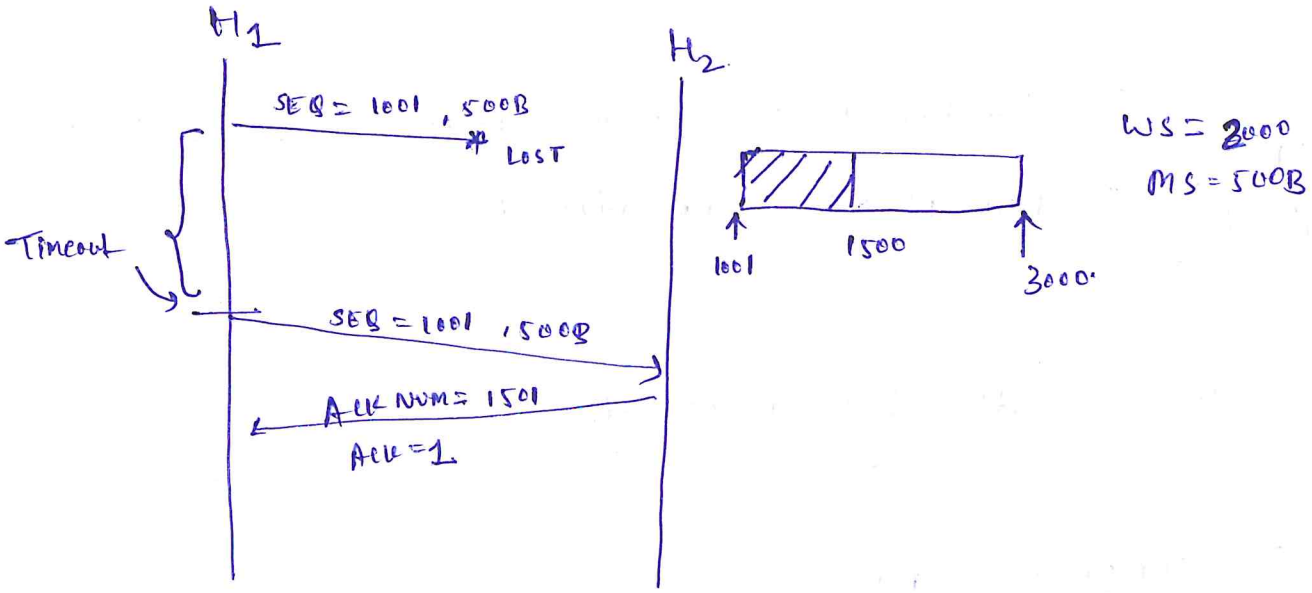
Expecting SEQ = 1000



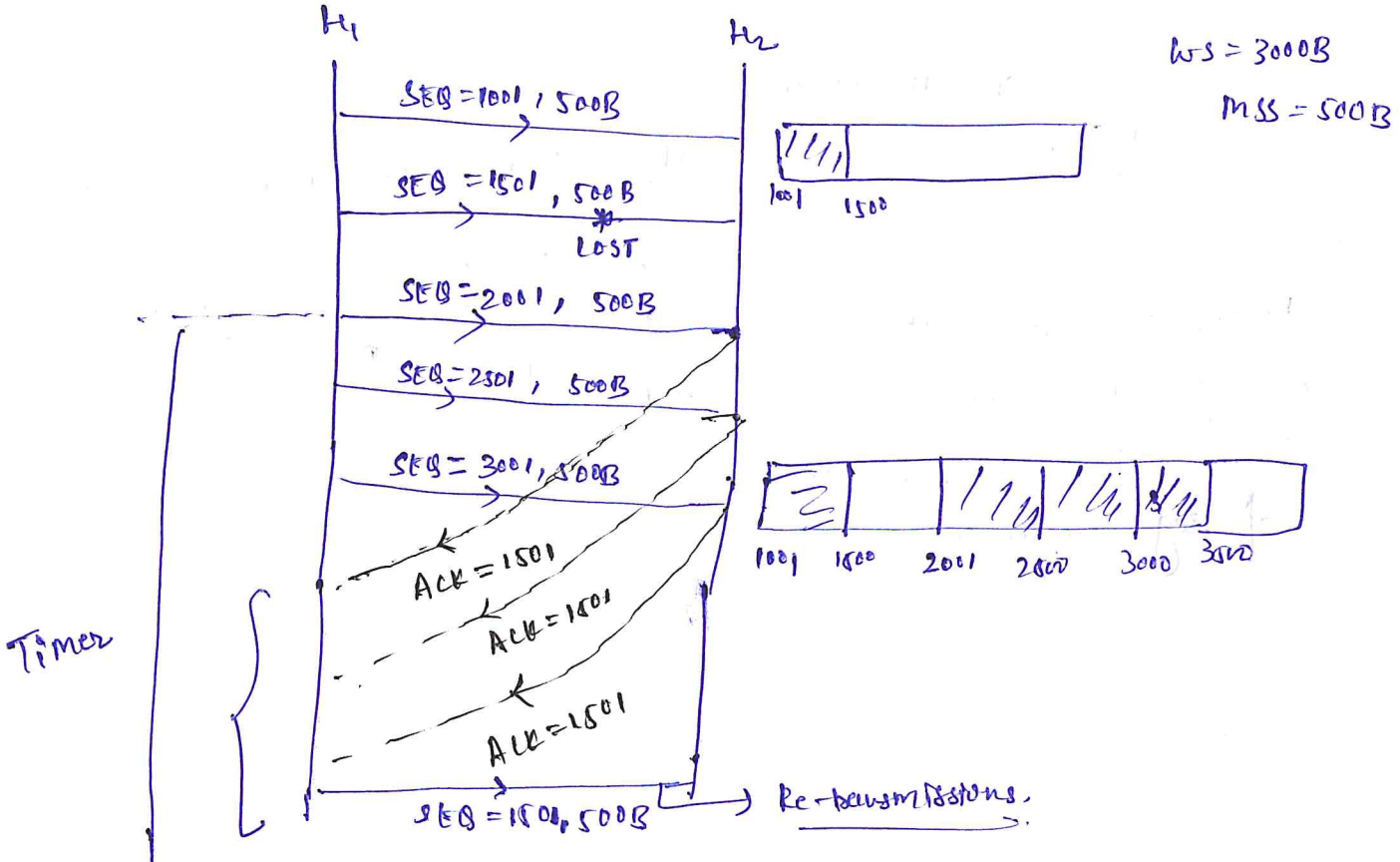
WS = 2000B

Here, we accept that out of bounds sequence number because belongs to the same window size.

Timeout - based Retransmissions :-



Dup Ack based / Early Retransmissions : (3 duplicate Ack)



→ In TCP we continue sending the segments don't wait for individual Ack's.

→ Sender can send out of order frames but receiver always send in-order ACK's in TCP.

→ After receiving three ACK's from sender knows that something has happen and it again sends the lost segment.

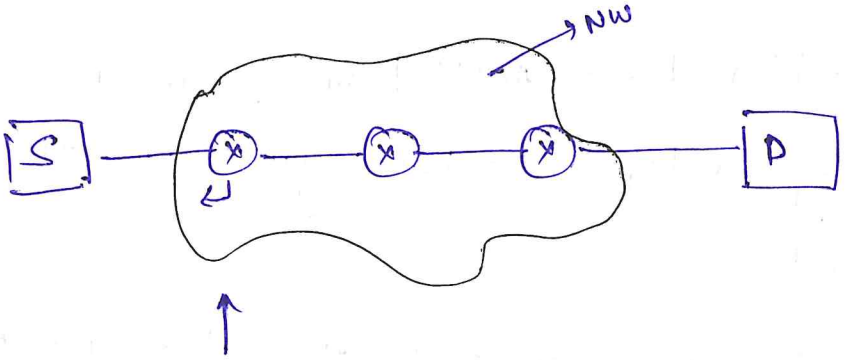
→ Before timeout sender sends the lost packet so called as early re-transmission.

→ TCP doesn't worries about individual ACK's because next ACK after re-transmission will be 3501.
This way TCP knows before that all frame successfully reached.

* If there is too many time outs in the TCP it tells there is strong congestion.

TCP Congestion control :

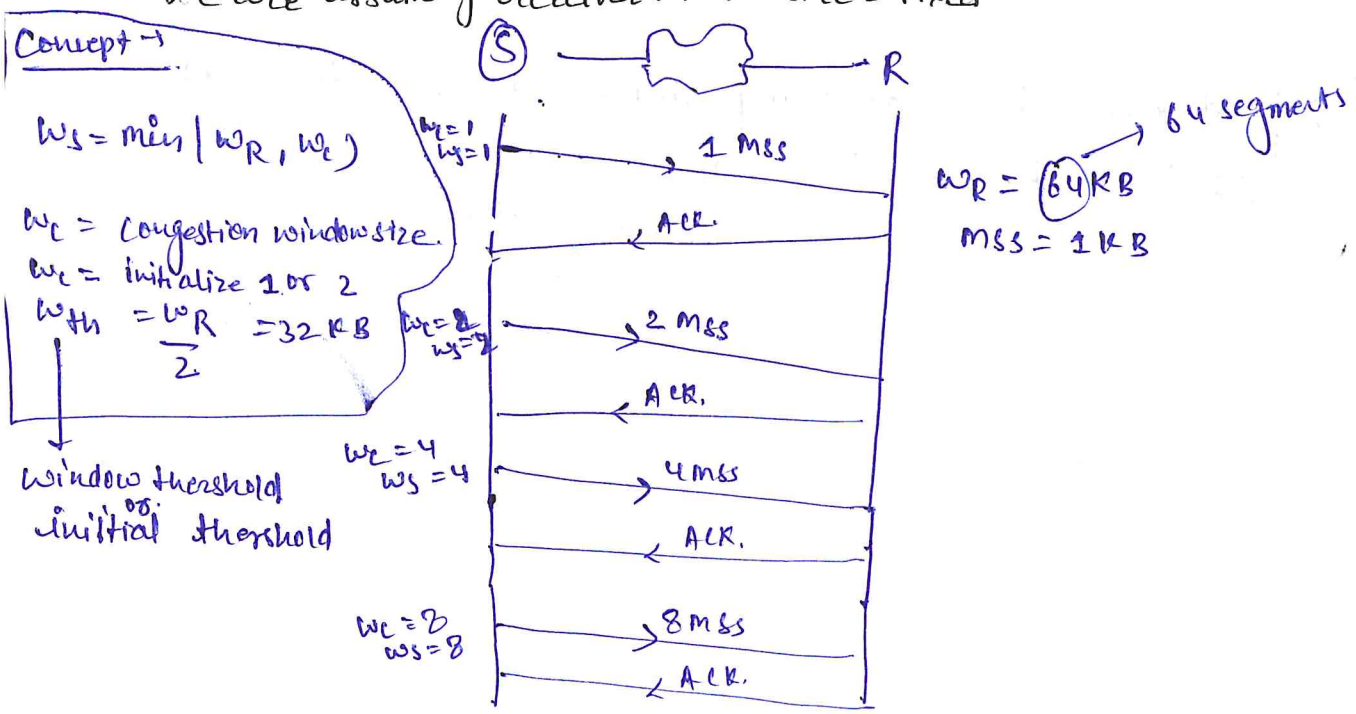
→ end-end congestion control



ICMP source Quench.
help us to know the congestion on single router

→ In TCP we are concerned about end-end congestion control.

We are assuming receiver window size = fixed



1 2 4 8 16 32. 33 34 ---- 64

Slow start phase.
Exponential growth
$\times 2$

After threshold go linearly.

congestion avoidance

We have two phases in congestion control:

(1) Slow start phase [grows exponentially upto threshold]

(2) Congestion avoidance phase [After threshold grows linearly]

→ No. of round trip in previous ex to reach 64.

$= 6 + 31 = 37$

6 → Slow start; 31 → congestion avoidance.

→ No. of round trip to complete 64 seg.

$= 6 + 32 = 38$

Algorithm:

(1) Slow start phase $w_c = 1$ or 2. (Initial congestion window)
↳ exponential growth.

(2) Congestion avoidance.
↳ linear growth in w_c

(3) Continuously detect congestion.

(a) Timeout ⇒ severe congestion ⇒ $w_{Th} = \frac{w_c}{2}$ & slow start

(b) 3-dup Ack ⇒ mild congestion ⇒ $w_{Th} = \frac{w_c}{2}$ & congesti avoida

Ex→ Initially

$W_R = 64$

So, threshold $= 64/2 = 32$

We perform

Let, Suppose Timeout

$W_c = 1, 2, 4, 8, 16, 32, 33, 34, 35, 36$

Slow start — Congestion avoidance — Sense congestion

So, New $T_H = 36/2 = 18$

$W_R = 64$

Now again Slow start:—

1, 2, 4, 8, 16, 18, 19, 20 → 3 duplicate Ack so, mild congestion

18 → 32 not possible so go upto threshold

Slow start — CA

So, new $T_H = 20/2 = 10$

$W_R = 64$

Now apply congestion avoidance.

10, 11, 12, 13, 14, 15, 16, 17 ..... 64, 64, 64

Congestion avoidance

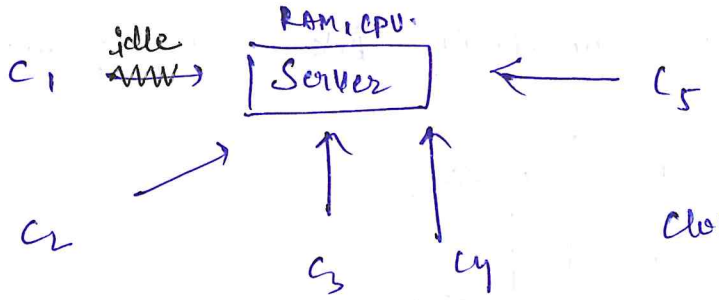
It can not go beyond 64 because receiver window size = 64

Timers:

(1) Time-wait → This timer we have studied in TCP state transition diagram.

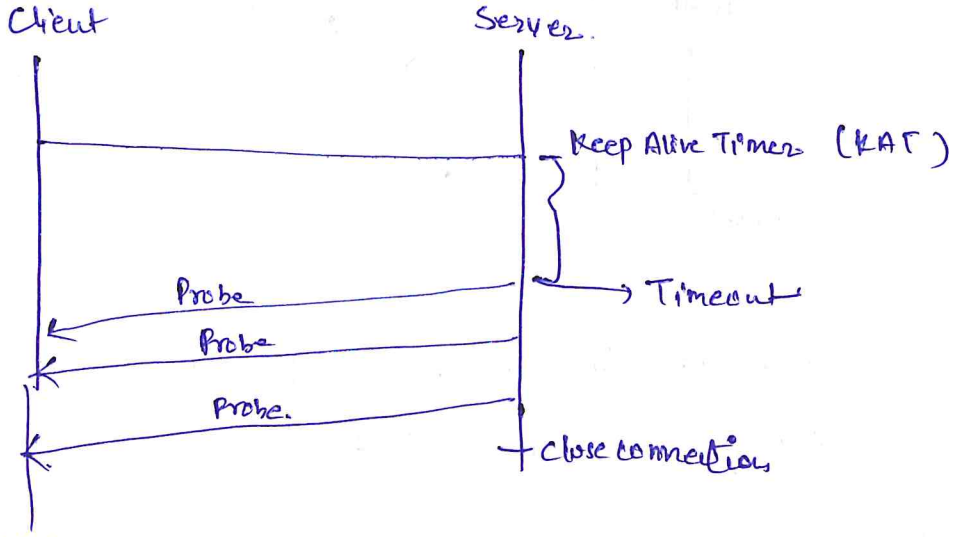
(2) Persistent timer → This we have studied in the flow control. If ws = 0B advertised by receiver then we start at persistent receiver.

(3) Keep alive timer →



Close idle connections.

Server is connected to multiple clients. C_1 is not sending any data such a connection called the idle connection. We want to terminate the idle connections



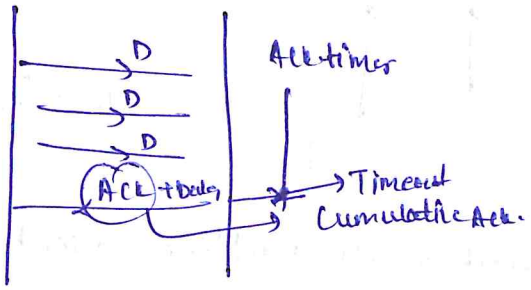
* When server are getting packets from the client it always

update the keep alive timer..

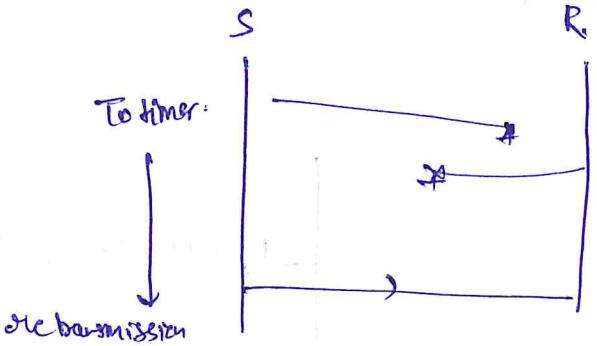
If for some duration of time if there is no packet send by client to server then KAT will time out

→ Before closing the connection servers sends three probe segment over a period of interval. If there is no response from the sender then client closes the connection.

(4) Acknowledgement Timer → Cumulative Ack and Piggybacked Ack



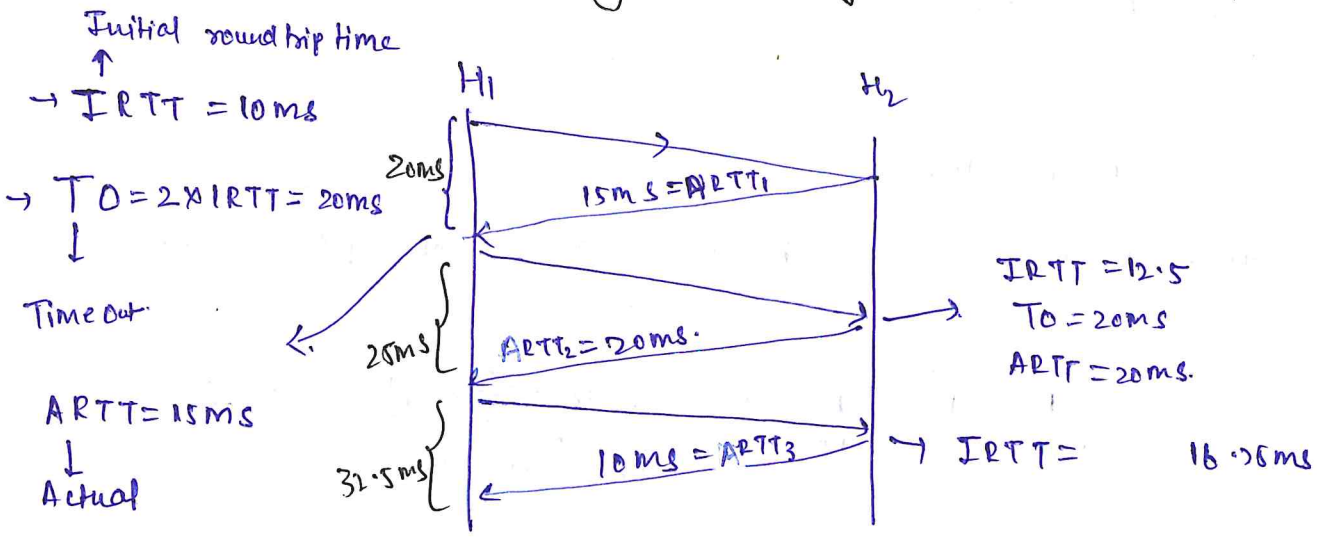
(5) Timeout timer :-

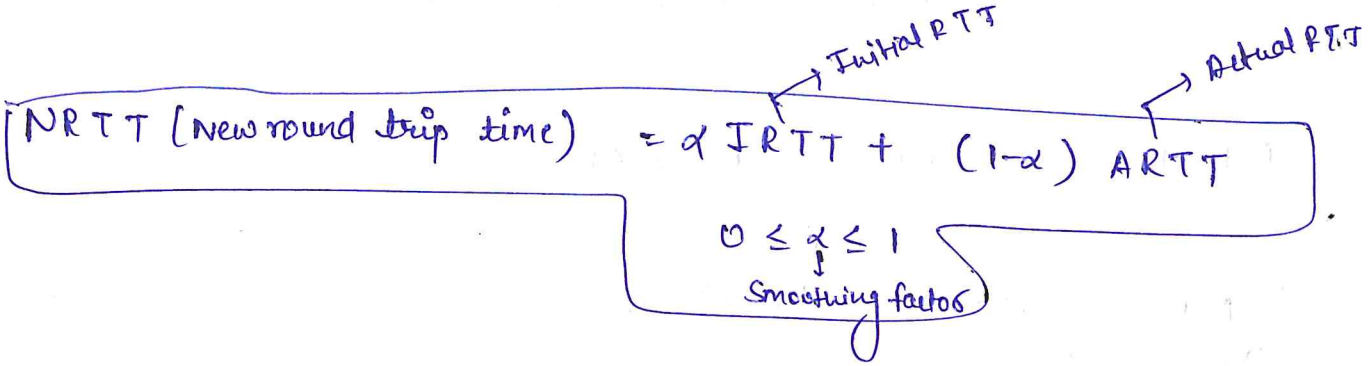


The time-out timer is used for re-transmissions if Ack from receiver is lost

* The timeout timer dynamically changes the value because of change in traffic

Timeout Timer setting (Basic Algo)





Let $\alpha = 0.5$

$NRTT_1 = 0.5 \times 10 + (1-0.5) \times 15ms$

$NRTT = 12.5\ ms$

New $TO = 12.5 \times 2 = 25\ ms$

$NRTT_2 = 0.5\ (12.5 + 20) = 16.25ms$

$TO = 32.5\ ms.$

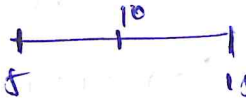
$NRTT_3 = 1/2\ (10 + 16.25) = 13.125ms$

$TO = 26.25\ ms$

Jacobson's Algorithm:

IRTT = 10 ms.

Deviation = ID = 5 ms ⇒ Interval estimation not a point estimation



⇒ The deviation of 5 ms in the interval.

Timeout = (4 × ID) + IRTT = 30 ms.

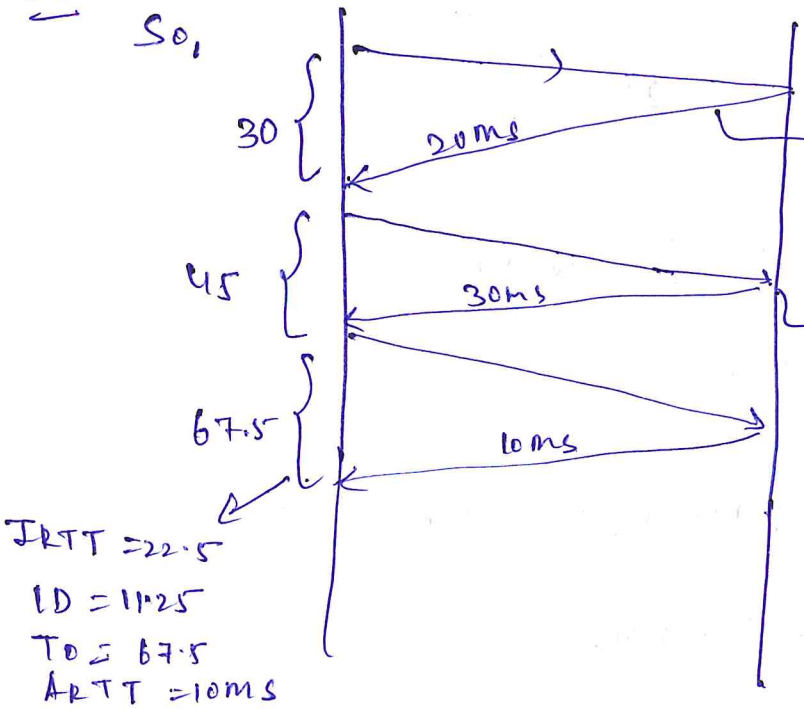
NRTT = α × IRTT + (1−α) ARTT

ND = α × ID + (1−α) AD
(New deviation)

AD (Actual deviation) = (IRTT − ARTT)

Ex →

So,



Timeout = 30
ARTT = 20
NRTT = 15 ms
ND = 7.5

Timeout = 45 ms
IRTT = 15 ms, ID = 7.5 ms
ARTT = 30 ms, NRTT = 22.5
ND = 11.25

IRTT = 22.5
ID = 11.25
TO = 67.5
ARTT = 10 ms
NRTT = 16.25 ms
ND = 11.875 sec

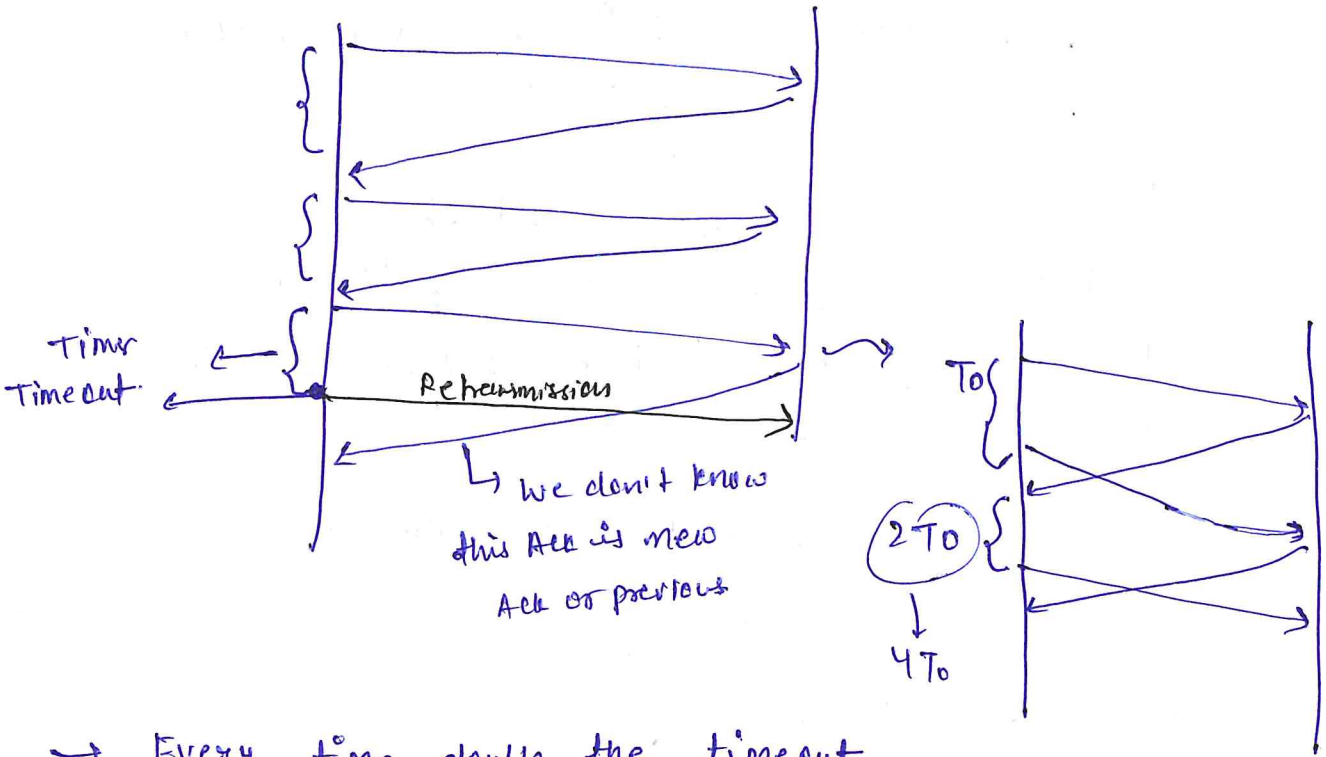
Karn's Modification

→ Whenever a situation arrives

ARTT > TO

* We have to retransmit because timer has already expired.

{ how to calculate NRTT as ARTT is not available.



→ Every time double the timeout when ARTT > TO

↓

Once ARTT < TO then follow the normal concept.

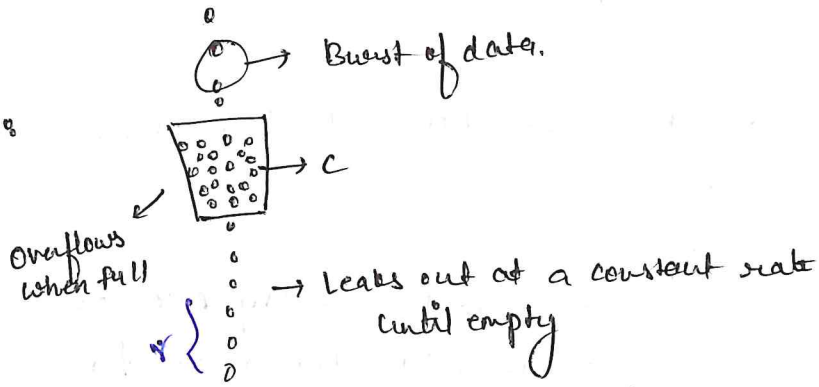
Congestion control : Traffic shaping

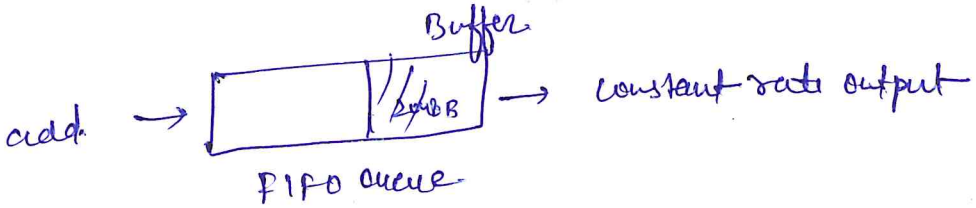
→ Parameter Negotiation : rate of data transfer

* We can negotiate the parameter of rate of data transfer

→ Sender maintains the agreed rate.

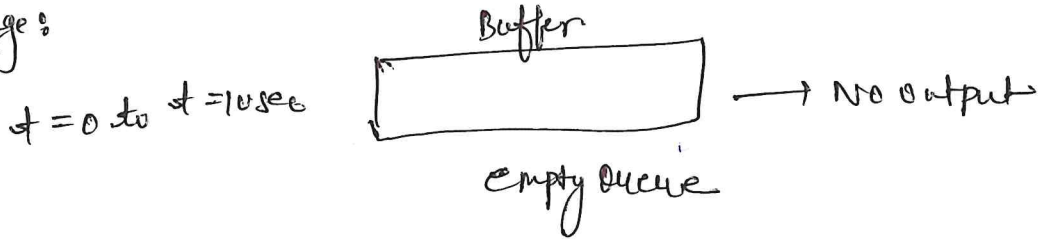
Leaky Bucket :



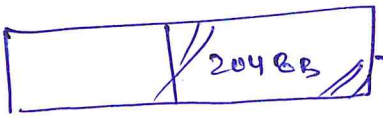


* The output rate is always constant
* If queue is full it will discard the packet / overflow.
* The adding rate might not constant.

Disadvantage :



Initially upto 10 sec there is no data in the Queue.



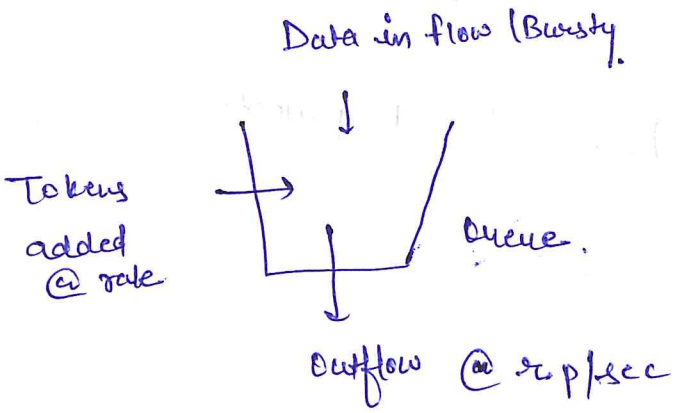
t = 11 sec
(Burst of data)

↳ 4 sec to output this data

T = 11 sec we get burst of data.

Initial 10 sec can not be used because we have no data that time.

Token Bucket :



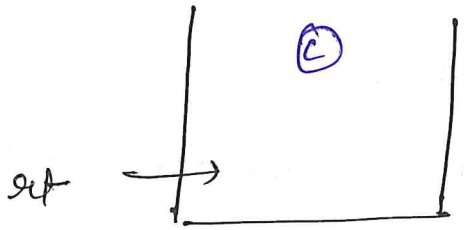
* Let suppose we are adding 5 tokens per second
* So, in 10 sec we have 50 tokens in the token bucket
* At 11 sec we got 4 packets in the bucket.

These 4 packet can outflow suddenly assigned with tokens. We don't have to wait one by one.

Bucket capacity ≥ Tokens in the bucket @ any time

Max no of packet in t sec.

0 ——— t



→ In t sec rt number of tokens will be added in this bucket.

→ Max no. of packet in t sec = C + rt (Initial C packets)

→ Max avg rate = $\frac{C + rt}{t}$ · p/sec → p = packet

UDP (User datagram protocol)

(1) UDP is connection less protocol. It does not uses byte stream like TCP

(2) Application layer protocol need few req/response packets.

e.g. DNS, BOOTP, DHCP, OSPF (Routing)

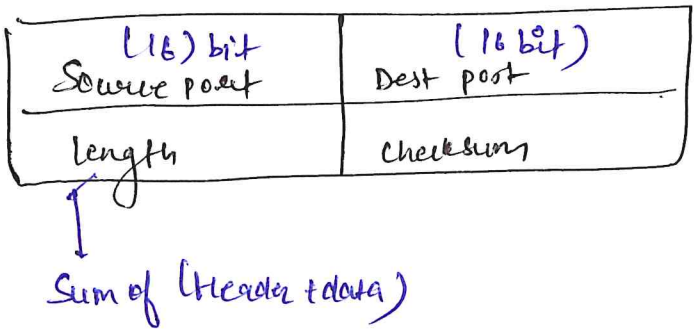
↳ There is no need of connection establishment overhead.

(3) Broadcasting and multicasting (many hosts).

↳ TCP has lot of overhead needed buffer

Use UDP for most of these operations.

UDP header →

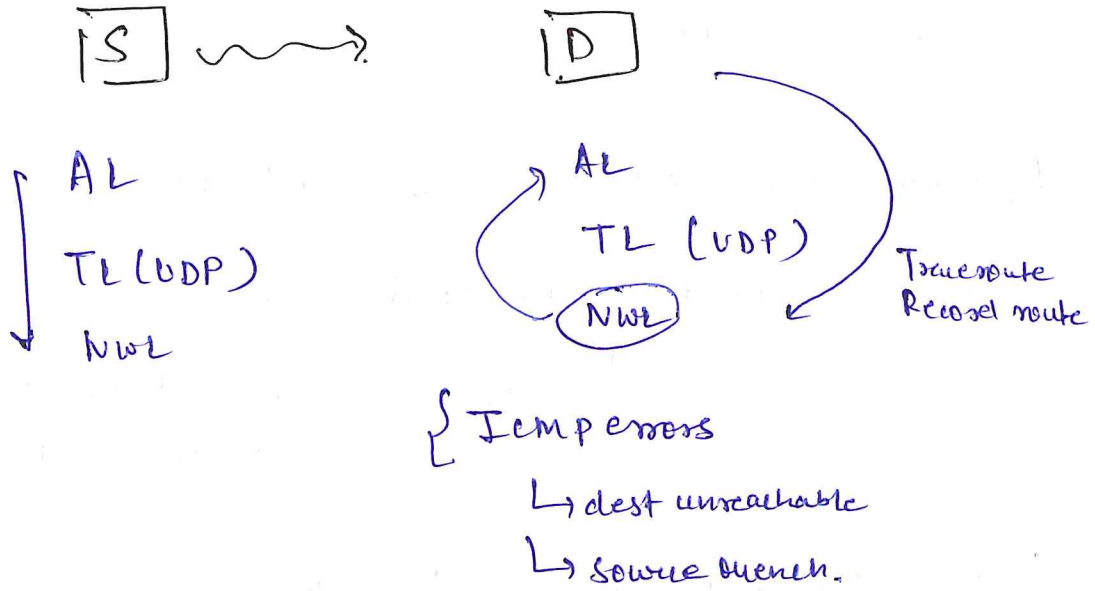


* $2^{16} - 1$ = max size of segment.

Checksum is computed = UDP-H + UDP data + Pseudo header

* For speed we don't compute checksum and often ly. it is disabled and all one's are filled.

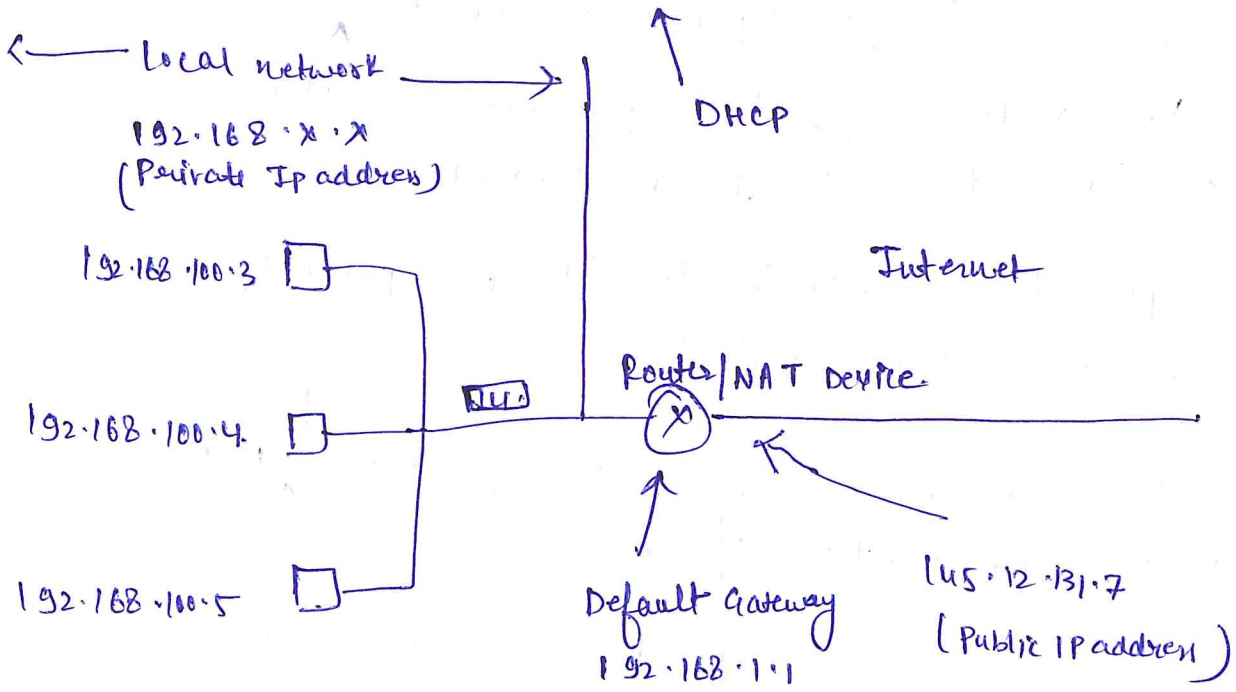
Additional task of UDP:



{ Icmp errors
↳ dest unreachable
↳ Source Quench.

* One of the main task of UDP to communicate key information from NWL to AL like Icmp errors.
* It also facilitates additional information from AL to network layer

Network Address Translation :- Task of network layer.



Private Ip range : 10.0.0.0 — 10.255.255.255 /8 (16777216 hosts)
172.16.0.0 — 172.31.255.255 /12. (1048576 hosts)
192.168.0.0 — 192.168.255.255 /16 (65536 hosts)

Translation table of NAT router :

Private Ip	Port
192.168.100.3	5000

* The NAT router converts the private Ip to public Ip while sending packet out in the network.
* The NAT router converts public Ip to private Ip when sends packet in the network.

* The another task of NAT router is whenever the system is sending a packet outside the network the NAT router assigns the post number with private Ip address in the table and after assigning it replace the private Ip to its own public Ip and send to the destination.

* The destination replies with public Ip address of router with assigned post number of router.

* Router forward the reply packet with changing the public Ip to private Ip with the assigned post number by looking in table.

Solved Problems:

Que: Consider a long lived TCP session with an end-to-end bandwidth of 1 Gbps = (10^9 bits-per second). The session starts with sequence number of 1234. The minimum time (in seconds, rounded to the closest integer) before that sequence number can be used again is? ____

Ans: Bw = 1 Gbps

For seq no = 32 bits

we have = 2^{32} B (data to transfer)



→ sequence bits goes in cyclic [ignore header]

So, we have to pass 2^{32} B to again come to 1234

$$\text{Total time} = \frac{2^{32} \times 8 \text{ b}}{10^9 \text{ bps}} = 34.35 \text{ sec} = 34 \text{ (Ans)}$$

closest integer

Que:

	Field.		length in bits
P.	UDP header Port no.	I.	48
Q.	Ethernet MAC addr	II.	8
R.	IPv6 next header	III.	32
S.	TCP header's sequence number	IV.	16

Match the following?

(A) P - III, Q - IV, R - II, S - I

(B) P - II, Q - I, R - IV, S - III

(C) P - IV, Q - I, R - II, S - III

(D) P - IV, Q - I, R - III, S - II

Ans → C.

UDP portno = 16bit

Ethernet mac addr = 48bit

IPv6 next header =

TCP header's sequence no = 32

Que: Consider the following statements regarding the slowstart phase of the TCP congestion control algorithm. Note that Cwind stands for the TCP congestion window and mss denotes the maximum segment size.

1. Cwnd increase by 2mss on every sucessful acknowledgement.
2. The Cwnd approximately doubles on every sucessful acknowledgement.
3. The Cwnd increase by 1mss every round trip time.
4. The Cwnd approximately doubles every round trip time.

Which one of the following is correct?

(A) Only (i) and (iii) are true
(B) only (i) and (iv) are true.
✓(C) only (iv) is true
(D) only (i) and (iv) are true

Ans: mss = maximum segment size

Cwnd = Congestion window.

1 → wrong it doubles in slow start phase

2 → It does not necessary for TCP to wait for ACK.

3 → wrong

4 → True, It approximately doubles every round trip time.

Que: Consider a TCP client and a TCP server running on two different machines. After completing data transfer, the TCP client calls close to terminate the connection and a FIN segment is sent to the TCP server. Server-side TCP responds by sending and Ack. which is received by the client-side TCP. As per the TCP connection state diagram (RFC-793), in which state does the client side TCP connection wait for the FIN from the server-side TCP?

1. LAST-ACK
2. TIME-WAIT
3. FIN-WAIT-1
4. FIN-WAIT-2

Ans: Ans: FIN WAIT-2 [Refer State transition diagram TCP]

Que: Consider a socket API on Linux machine that supports UDP socket. A connected UDP socket is a UDP socket on which connect function has already been called. which of the following statements is/are correct?

I. A connected UDP socket can be used to communicate with multiple peers simultaneously.

II. A process can sucessfully call connect function again for an already connected UDP socket.

(A) I only

(B) II only

(C) Both I and II only

(D) Neither I nor II



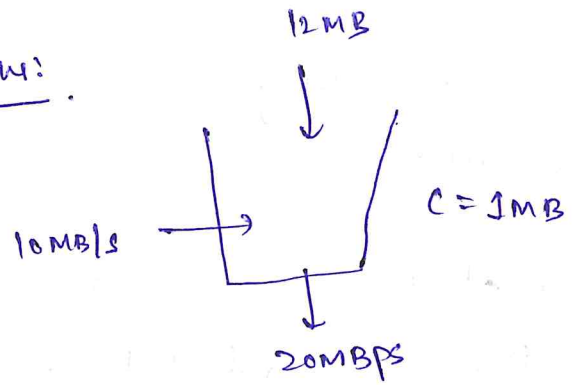
Ans:

I → In a UDP socket when it is connected we can not communicate to multiple peers because every peer have different socket address.

II. Yes, UDP is connectionless service we can call connect function again to serve a different socket. There is no concept of closing or terminating a connection.

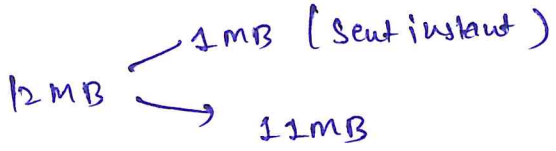
Que: For a host machine that uses the token bucket algorithm for congestion control, the token bucket has a capacity of 1 megabyte and the maximum output rate is 20 megabytes per second. Tokens arrive at a rate to sustain output at a rate of 10 megabytes per second. The token bucket is currently full and machine needs to send 12 megabytes of data. The minimum time required to transmit the data is ____ seconds?

Ans:



In 12MB data because of 1MB token.

1MB data will send instantaneously.



1 sec = 10MB tokens generated.

1.1 Sec = 11MB tokens generated

The minimum time to transmit is 1.1 sec.

Que: Suppose two hosts use a TCP connection to transfer a large file. Which of the following statements is/are false with respect to the TCP connection?

1. If sequence number of a segment is m, then the sequence number of the subsequent segment is always m+1.
2. If the estimated round trip time at any given point of time is t sec, the value of the retransmission timeout is always set to greater than or equal to t sec.

3. The size of the advertised window never changes during the course of the TCP connection.

4. The number of unacknowledged bytes at the sender is always less than or equal to the advertised window.

(A) 3 only

(B) 1 and 3 only

(C) 1 and 4 only

(D) 2 and 4 only.

Ans: (1) In TCP we uses the the byte stream the. Sequence number of next segment is [m + how many data bytes.] → false

(2)(3) The size of advertised window do change so this statement false.

Ans → B.

Que: Identify the correct order in which a server process must invoke the function calls accept, bind, listen and recv according to UNIX socket API.

(A) listen, accept, bind, recv

(B) bind, listen, accept, recv

(C) bind, accept, listen, recv

(D) accept, listen, bind, recv

Ans → First bind the port number, listen for connections, accept and then recv.

Que: Host A sends a UDP datagram containing 8880 bytes of user data to host B over an Ethernet LAN. Ethernet frames may carry data up to 1500 bytes (i.e MTU = 1500 bytes). Size of UDP header is 8 bytes and size of IP header is 20 bytes. There is no option field in the IP header. How many total number of IP fragments will be transmitted and what will be the contents of offset field in the last fragment?

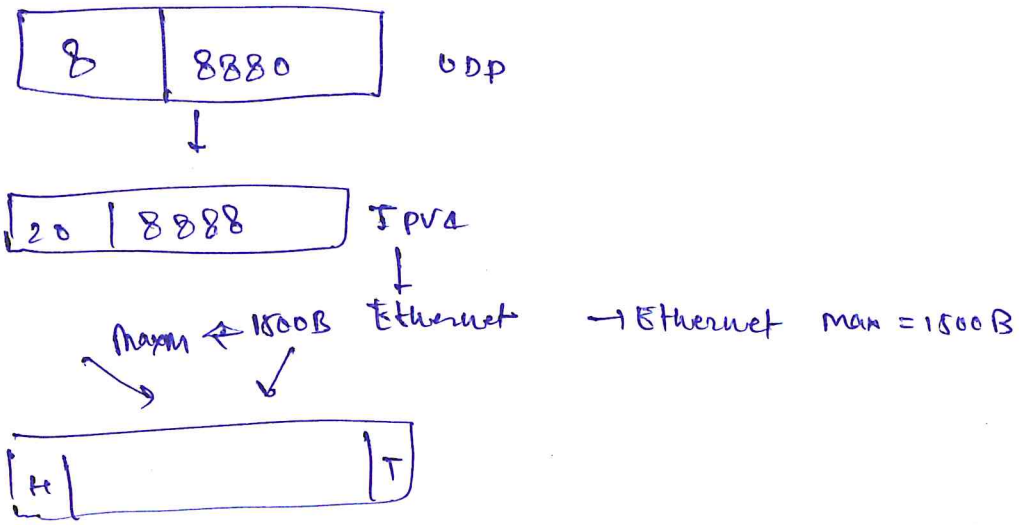
(A) 6 and 925

(B) 6 and 7400

(C) 7 and 1110

(D) 7 and 8880

Ans:



So, $\frac{20 \mid 1480}{\text{Header}\quad\text{Max possible}} = \frac{8888}{1480} = 7$ fragments.

Fragments $= \frac{1480 \times 6}{8} = 1110$ (6 in subscript, 8 → bits)

Que: Consider the following statements.

I. Tcp connection are full duplex.

II. Tcp has no option for selective acknowledgement.

III. Tcp connection are message streams.

(A) Only I correct

(B) only I and IV are correct.

(C) only II and III are correct.

(D) All I, II, III are correct

II → Using option field we can give selective Ack's.

III → They are byte streams